

Foundations & the Responsible Engineer

How software engineers actually work, which myths to unlearn, and which rules protect the people who use software, or are affected by it. In the Studio you apply all of it to your own product.

Learning outcome

LO5 (Learning Outcome 5) — Recognise the professional, ethical and social responsibility of a Software Engineer.

Topic 1 — Foundations; Professional, Ethical and Social Responsibility

Today's five lessons



1 How engineers work

essence of practice, seven principles



2 Myths we unlearn

and Brooks's law



3 People and stakeholders

teams, roles, the power-interest grid



4 The Code of Ethics

eight principles, professional status



5 Social responsibility

safety, privacy, fairness, accessibility, sustainability

Today: two questions, one hour in four parts

Last Tuesday: teams formed, product chosen, vision card written (Block 1).

TWO QUESTIONS FOR TODAY



Once you know what you want to build, how does a software engineer actually work on it?

Lessons 1–2



What does a software engineer owe to the people who use, or are affected by, that software?

Lessons 3–5

ONE HOUR IN FOUR PARTS, EACH WITH AN ACTIVE BREAK



0:00

Lesson 1 — The essence of practice and seven principles



0:11 flash-cards



0:15

Lesson 2 — Software myths and Brooks's law



0:26 myth or reality?



0:30

Lessons 3–4 — People, stakeholders, the Code of Ethics



0:41 visualize-it



0:45

Lesson 5 — Social responsibility



0:56 Zen-break

Then 1:00 break · 1:10 Studio (100 min) · 2:50 wrap-up, exit ticket

The essence of practice (1/2): four steps



You receive a programming assignment. What is the first thing you do?

For a product that other people depend on, typing code at once usually fails: the code solves a problem that nobody defined clearly. Engineering practice exists to prevent that.



Understand the problem

In a project: communication and analysis



Plan a solution

In a project: modeling and design



Carry out the plan

In a project: construction (code and test)



Examine the result

In a project: testing and quality assurance

Examining the result often reveals a new problem → back to step 1: **practice is iterative.**

Source: George Pólya, *How to Solve It* (1945). Our textbook uses the same four steps to describe the **essence of practice**.



The essence of practice (2/2): questions at each step

1 Understand the problem

- Who has a stake in the solution?
- What data, functions and behaviour are required?
- Can it be divided into smaller problems?
- Can it be represented as a diagram or a model?

In a project: communication and analysis

2 Plan a solution

- Have I seen a similar problem before?
- Is there a solution, component or pattern to reuse?
- Can I define sub-problems and solve each one?
- Can I describe a solution that leads to an implementation?

In a project: modeling and design

3 Carry out the plan

- Does the code follow the design?
- Is each part correct?
- Has each part been reviewed and tested?

In a project: construction (code and test)

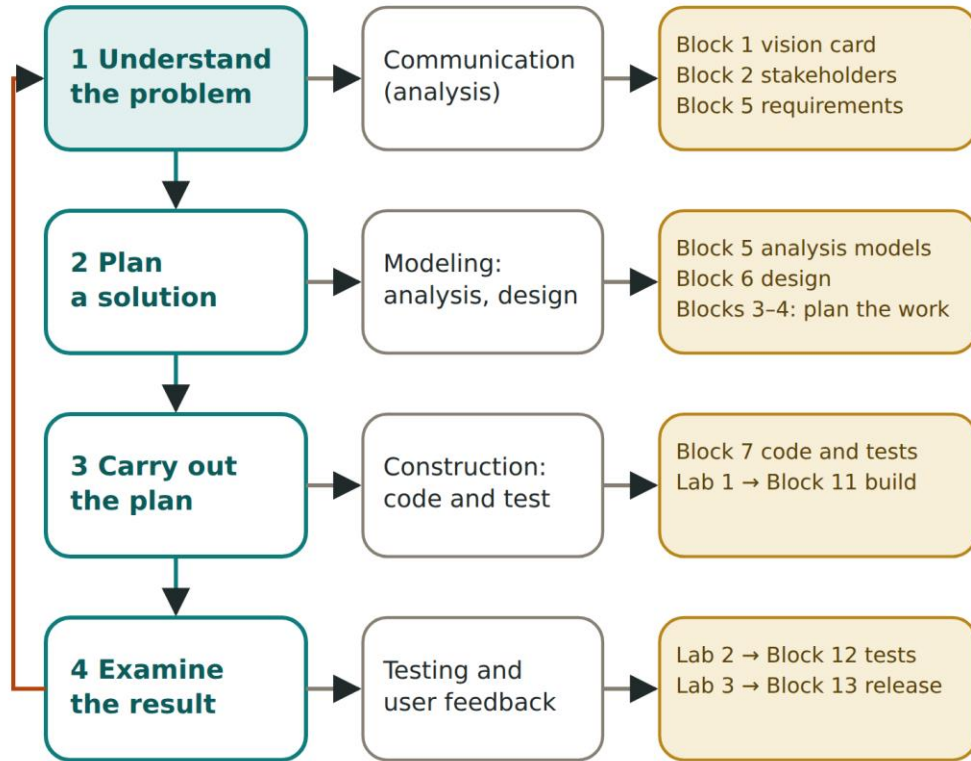
4 Examine the result

- Can each part of the solution be tested?
- Does the result produce the required data, functions and behaviour?
- Did it reveal a new problem? Then back to step 1.

In a project: testing and quality assurance

Where the four steps appear in your project

Pólya's four steps → framework activities → your Blocks



Examining the result often reveals a new problem → back to step 1.

The four steps repeat: practice is iterative.

From Figure 1 of the module (download it as .svg or .png in Lesson 1).

The four steps map onto the five **framework activities** of Class 1:

- 1 Understand** communication (analysis) — Blocks 1, 2 and 5
- 2 Plan** modeling: analysis and design models — Blocks 5 and 6
- 3 Carry out** construction (code and test) — Block 7, Lab 1
- 4 Examine** testing, plus the feedback you receive after deployment — Labs 2 and 3

The project planning of Blocks 3 and 4 organizes the work around the models. Examining the result can send you back to step 1: in your project, a failed test or a user complaint starts a new cycle.

Hooker's seven principles (1/2)

A **principle** is a general rule that guides decisions when no specific procedure tells you what to do. David Hooker (1996) proposed seven for software engineering practice.

1 The Reason It All Exists

A system is worth building only for the **value it gives its users**.

Ask: *Does this give a user something of real value? If not, do not do it.*

2 KISS (Keep It Simple, Stupid)

Choose the **simplest design that still meets every requirement**. Simple is not careless: it is easier to understand, test and change.

Ask: *Is there a simpler design that still meets the requirement?*

3 Maintain the Vision

One clear, shared idea of the product gives **conceptual integrity** (Brooks): the system looks as if one mind designed it.

Ask: *Does this fit our vision card (Block 1)?*

4 What You Produce, Others Will Consume

Someone else will read, use, maintain or test what you produce: write for that person.

Ask: *Could a new team member understand this without us?*



Hooker’s seven principles (2/2)

5 Be Open to the Future

Requirements change during a long service life: design so the system can be extended, ask “what if” questions, and avoid decisions that block likely changes — without building them now.

6 Plan Ahead for Reuse

Reuse rarely happens by accident: design the interface and the documentation of a reusable component for reuse from the start.

7 Think!

Think the problem through before you act: the result is almost always better, and a mistake you still make is one you can learn from.

USE THEM AS QUESTIONS WHEN YOU DECIDE

When ...	ask ...
someone proposes a feature	The Reason It All Exists
you choose a design	KISS (Keep It Simple, Stupid)
a new idea does not fit the vision card	Maintain the Vision
you write a document or a function	What You Produce, Others Will Consume
you define an interface	Be Open to the Future
a component could be useful elsewhere	Plan Ahead for Reuse
before any of that	Think!

Chapter 6 (this week’s reading) adds more:

process principles (quality at every step, adapt, build an effective team, manage change, assess risk) and practice principles (divide and conquer, abstraction, consistency, modularity, patterns, maintainability).



Example — QueueLess (1/2): understand, then plan

Task Reduce the time students stand at the QueueLess pickup counter, using the four steps and Hooker’s principles.

Relation to the lecture the essence of practice applied to a real complaint; principles eliminate options.

STEP 1 · UNDERSTAND

“I ordered at noon, I walked to the counter when the app told me to, and I still waited fifteen minutes.”

- 16 orders placed between 11:55 and 12:07.
- The app says “ready in about 4 minutes”, but the kitchen needs at least 3 minutes per order and finishes one every 2 minutes, in **FIFO** (First In, First Out) order.

8.8 min
mean wait

17 min
longest wait

19%
within 2 minutes

Success criterion (written first): at least 90% of students wait 2 minutes or less.

STEP 2 · PLAN: FOUR OPTIONS

A

Tell each student the real ready time, computed from the kitchen queue

KISS (Keep It Simple, Stupid); it serves every user.

KEPT

B

Predict ready times with a machine-learning model

Estimator placed behind a replaceable interface (Be Open to the Future).

NOT NOW

C

Open a second pickup counter

The kitchen rate does not change, so the waiting stays.

NO EFFECT

D

Sell a paid priority pass

Conflicts with The Reason It All Exists: quick pickup for every student.

REJECTED



Example — QueueLess (2/2): carry out, examine, Think!

Step 3 · Carry out the app computes each order's ready time from the kitchen queue and tells each student when to come.

STEP 4 · EXAMINE

8.8 → 0.5

minutes: mean wait at the counter

19% → 94%

within 2 minutes: the criterion ($\geq 90\%$) is met

12.8 → 13.3

minutes from order to meal: not improved



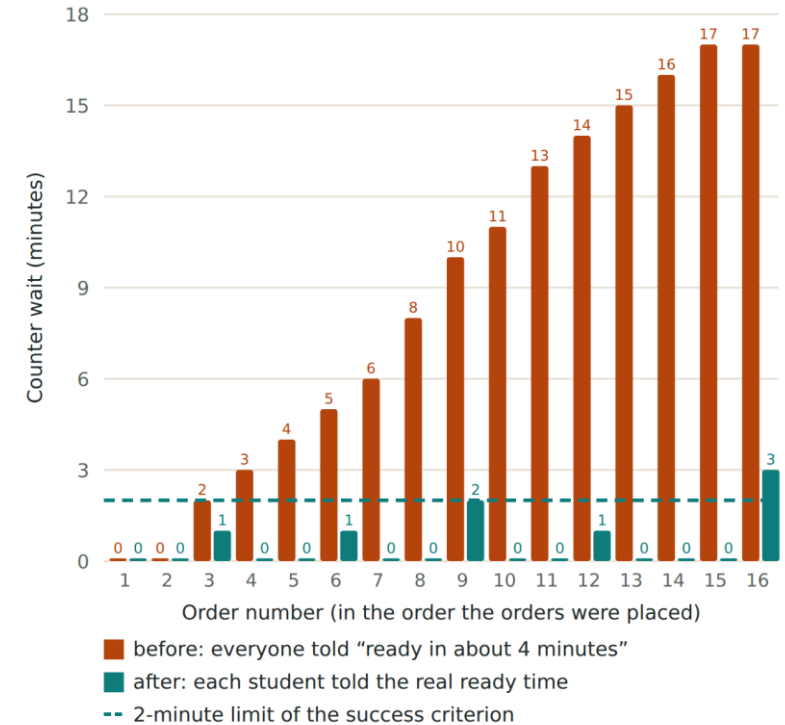
Think! Students who arrive late add their own delay, and the kitchen is still the slowest step: the waiting moved elsewhere.

That is a new problem → the cycle starts again at step 1.



Demo 1 runs this example step by step: pause at any point and read every calculation in the trace window.

QueueLess: counter wait of each order



Mean counter wait 8.8 → 0.5 minutes · longest 17 → 3 minutes
Within 2 minutes: 19% → 94% (criterion: at least 90%, met)
Order-to-meal time 12.8 → 13.3 minutes:
the kitchen is still the slowest step (new problem).

From Figure 1: rust = before, teal = after the change.

Lesson 1 after class: watch, run, reuse



Watch after class

“Got a wicked problem? First, tell me how you make toast”

TED (Technology, Entertainment, Design) — speaker Tom Wujec · 8:51

Why: people understand a problem better when they draw it; a group reaches one shared model.

Open: [YouTube](#) · [ted.com](https://www.ted.com)

Optional extra: “Polya’s Four Steps in Problem Solving (1.3)” — Professor Monte (length not published). [YouTube](#)



Run the demo

Demo 1 — The practice cycle on the QueueLess pickup data

- Start, pause, step and restart; every step is listed in the trace window.
- A comment line before every line of code.
- Figure 1 (.svg, .png) and the 16 orders (.csv).

Open it in the module (Lesson 1) or download the .html file from the Downloads section.



Comes back in your project

- **Block 4** (Oct 18): the success criterion as a Given / When / Then acceptance criterion
- **Block 5** (Oct 25): a measurable performance **NFR** (Non-Functional Requirement)
- **Block 6** (Nov 1): an **ADR** (Architecture Decision Record) for option A and a replaceable ready-time estimator
- **Block 7** (Nov 8): unit tests for the ready-time formula, with the boundary at exactly 2 minutes
- **Lab 2** (Dec 8 → Block 12): system test, **UAT** (User Acceptance Testing) scenario, performance micro-test

In the module (Lesson 1): five flash-cards and a self-check quiz — **formative, not graded**. Scan every link before opening it ([VirusTotal](#)).



Active break: flash-cards

0:11–0:15 · 4 minutes · in pairs



Partner A shows the front of each card; partner B names the principle and gives **one example from your own product**.

After two minutes, swap roles. Cards you missed go to the top of your home deck.

CARD 1 OF 5	CARD 2 OF 5	CARD 3 OF 5	CARD 4 OF 5	CARD 5 OF 5
The Reason It All Exists	KISS (Keep It Simple, Stupid)	Maintain the Vision	Be Open to the Future	Plan Ahead for Reuse
front of the card	front of the card	front of the card	front of the card	front of the card

The two other principles — “What You Produce, Others Will Consume” and “Think!” — are in the reference table under Figure 1 of the module.



What a software myth is



A **software myth** is a widely held belief about how software is built that **sounds reasonable but is contradicted by evidence and experience**. Because it sounds reasonable, people act on it.



Management myths

held by managers: standards manuals, adding people, outsourcing



Customer myths

held by the people who order the software: vague goals, easy changes



Practitioner myths

held by developers themselves: done when it runs, quality only at run time

Why do myths survive? Each one is true under some condition.

- Adding workers does shorten a job made of independent parts that need no communication.
- Changing a document is cheap when nothing else depends on it yet.

Method: find the condition under which the myth would be true, then check whether your project meets it. Software projects rarely do.

Management myths

MANAGEMENT MYTH 1

“Our company has a standards manual, so the team already knows how to build software well.”



REALITY

A manual helps only if people **know it exists, actually use it**, and it **reflects current practice**. Many manuals are outdated, incomplete or ignored.

Better question: do we follow a process that works, and do we improve it?

MANAGEMENT MYTH 2

“When we fall behind, hiring extra programmers will get us back on schedule.”



REALITY

The most famous myth — it has a name: **Brooks’s law**.

Adding people to a late project makes it later. Why? Three mechanisms, on the next slide.

The third management myth (outsourcing) comes after Brooks’s law.



Brooks's law

“Adding manpower to a late software project makes it later.”

Fred Brooks, *The Mythical Man-Month* (1975)
He managed the **OS/360** (Operating System/360) project at **IBM** (International Business Machines) in the 1960s; **ACM** (Association for Computing Machinery) Turing Award, 1999.



1 · Ramp-up time

A newcomer must learn the product, the code, the tools and past decisions. They produce little at first, and the experienced members who teach them produce less.



2 · Communication overhead

Every pair of people is a potential path:
 $n(n-1)/2$. $4 \rightarrow 6$ paths · $6 \rightarrow 15$ · $8 \rightarrow 28$.
The work does not grow; the coordination does.



3 · Limited divisibility

Sequential tasks cannot be split: you cannot test a feature before it is built, or build the report screen before the team agrees on its data.

An observation with conditions. People who join early, with independent tasks, before the project is late, can help. Once it is late, ramp-up and coordination cost the most when the least time is left.

One more management myth, and two customer myths

MANAGEMENT MYTH 3

“Once another company is building it for us, we no longer need to manage the project.”



REALITY

A team that cannot plan, track and accept its own work has the same problems with a contractor. **The client still owns the requirements, the acceptance tests and the risk.**

CUSTOMER MYTH 1

“We know roughly what we want, so programming can start now and the details can be settled later.”



REALITY

Vague objectives cause rework. The team needs a clear description of functions, behaviour, interfaces, constraints and validation criteria, reached through continuous communication (your Block 5).

CUSTOMER MYTH 2

“Changing software later is easy, because code can always be edited.”



REALITY

The cost of a change depends on **when** it is made: after design or release, the designs, code, tests and documents built on the old requirement must change too (Class 1: the cost-of-change curve).



Practitioner myths

PRACTITIONER MYTH 1

“As soon as the program works, the project is finished.”



REALITY

A large share of the total effort, often more than half, comes **after the first release**: corrections, adaptations, improvements (maintenance, in November).

PRACTITIONER MYTH 2

“There is no way to judge quality until the program runs.”



REALITY

A **technical review** assesses requirements and designs before any code exists, and finds defects when they are cheapest to fix.

PRACTITIONER MYTH 3

“A running program is the only thing a project needs to deliver.”



REALITY

The program is one part of a configuration that also includes **models, documents, plans and tests** — what lets others maintain and extend it.

PRACTITIONER MYTH 4

“Software engineering only produces extra documents, and they slow the team down.”



REALITY

Software engineering is about **quality**: less rework means faster delivery. Agile methods (Class 4) keep the documentation that is needed.



Example — ClubLend (1/2): a late team in numbers

Task Decide how a late ClubLend team should respond — add developers, or negotiate scope — by computing the finish day of each option.

Relation to the lecture tests the management myth against Brooks's law: ramp-up, $n(n-1)/2$ paths, limited divisibility.

4

people in Sprint 2

60

story points left

18

working days to release

0.8

points per person per day

6

communication paths ($n = 4$)

0.05

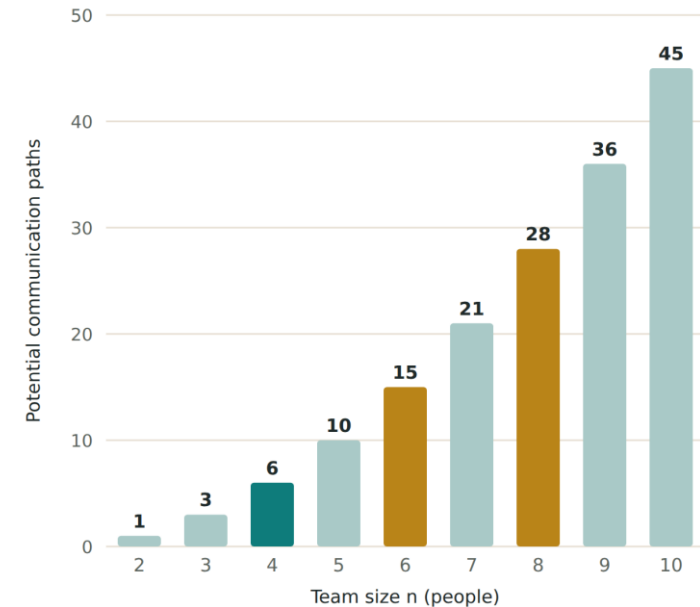
points lost per path per day

Team output per day

$$4 \times 0.8 - 6 \times 0.05 = \mathbf{2.9 \text{ points per day}}$$

$60 \div 2.9 \approx \mathbf{20.7 \text{ days}}$ against an 18-day deadline: about 2.7 days late.

Communication paths = $n(n-1)/2$



Teal: ClubLend today (4 people, 6 paths).

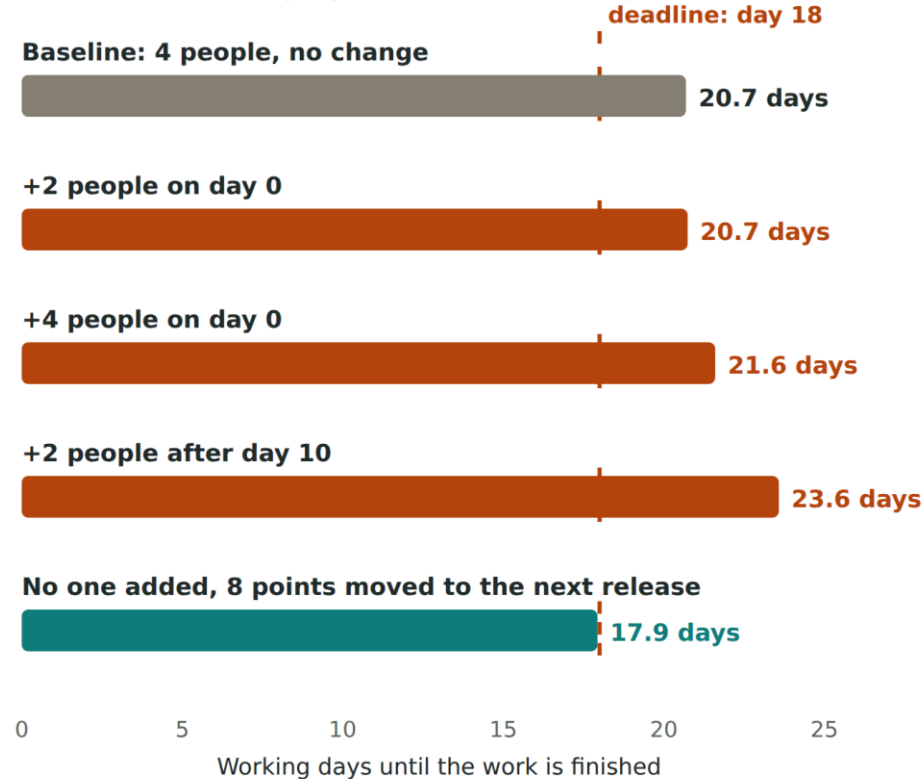
Amber: +2 people (15 paths) and +4 people (28 paths).

From Figure 2: paths grow faster than the team.



Example — ClubLend (2/2): add people or cut scope?

ClubLend finish day by scenario (teaching model)



Model: 60 points; 0.8 points per productive person per day;
10-day ramp-up; 0.4 points of mentoring per newcomer per day;
0.05 points per communication path per day. Teaching values.

From Figure 2 (teaching model; change every value in Demo 2).

Model: a newcomer needs **10 days of ramp-up** with no output and costs the team **0.4 points per day** of mentoring.

+2 people on day 0	≈ 20.7 days	no gain
+4 people on day 0	≈ 21.6 days	later
+2 people after day 10	≈ 23.6 days	much later
8 low-priority points moved to the next release	≈ 17.9 days	on time



The professional response

Report the delay early, re-estimate, and negotiate scope or schedule with the client. Hiding the delay, or adding people without analysis, makes it worse — and reporting it is a duty in the Code of Ethics (Lesson 4).



Lesson 2 after class: watch, run, reuse



Watch after class

“The Mythical Man-Month: a 9-minute book summary” (title shortened)

SnapTale · about 9 minutes (stated in its title)

Open: [YouTube](#)

“182. Lessons From ‘The Mythical Man-Month’”

THUNK · length not published

Open: [YouTube](#) · thunkshow.com

Compare their examples with the ClubLend numbers.



Run the demo

Demo 2 — Brooks’s law simulator for the late ClubLend team

- Choose the people added, the day they join and the scope removed.
- Watch the burndown line and the communication network grow, day by day.
- Figure 2 and three data files (.csv).

Open it in the module (Lesson 2) or download the .html file from the Downloads section.



Comes back in your project

- **Block 4** Sprint Plan (Oct 18): your real velocity; sprints needed = remaining points ÷ velocity (also in Part B of the Mid-Term)
- **Lab 1** (Dec 1 → Block 11): points done per session; compare your burndown with this baseline
- **Lab 3** (Dec 15 → Block 13): scope decisions recorded; release notes list “planned for the next release”
- **Block 13**: deferred stories start the maintenance backlog

In the module (Lesson 2): five flash-cards and a self-check quiz — **formative, not graded**. Scan every link before opening it ([VirusTotal](#)).



Active break: myth or reality?

0:26–0:30 · 4 minutes · whole class



Vote on each statement: **thumbs up** = **reality**, **thumbs down** = **myth**. Then the answer appears.

1

“If the project is late, adding two developers will bring it back on schedule.”

MYTH

Brooks’s law

2

“A review of the design document can find defects before any code exists.”

REALITY

reviews work on any work product

3

“Changing a requirement after release costs about the same as changing it while writing the requirements.”

MYTH

the cost grows with every work product that depends on it

This week, listen to your team: “we will write the requirements later”, “we can add someone”, “we will test at the end”. Name the myth; replace it with the reality.



Why people come first

Software is built by people and for people. Most project failures have **human causes**: unclear communication, poor coordination, and expectations that nobody wrote down.

TRAITS OF EFFECTIVE SOFTWARE ENGINEERS (CHAPTER 5)



Individual responsibility

for their own work



Awareness

of team and stakeholder needs



Honest

about flaws in their own designs



Constructive

about flaws in the designs of others



Resilient

under pressure



Fair

a strong sense of fairness



Attentive to detail

in every work product



Pragmatic

adapt the method to the situation

Constantine's four team paradigms

Teams have a structure, whether or not anyone designed it. Larry Constantine described four paradigms:



Closed paradigm

Classic chain of command.

Works well when projects that resemble earlier ones

Weakness new ideas are less likely



Random paradigm

Loosely organized; relies on each member's initiative.

Works well when innovation or a technical breakthrough is needed

Weakness struggles with orderly, repetitive work



Open paradigm

Some control plus innovation: collaborative work, a lot of communication, consensus decisions.

Works well when complex problems that need order and new ideas

Weakness can be less efficient; needs time to decide



Synchronous paradigm

Natural division of the problem: members work on separate pieces and communicate little.

Works well when the problem divides cleanly into independent parts

Weakness integration problems if the parts are not independent

Agile teams and your rotating roles



Self-organizing

The team **decides how to do its own work.**



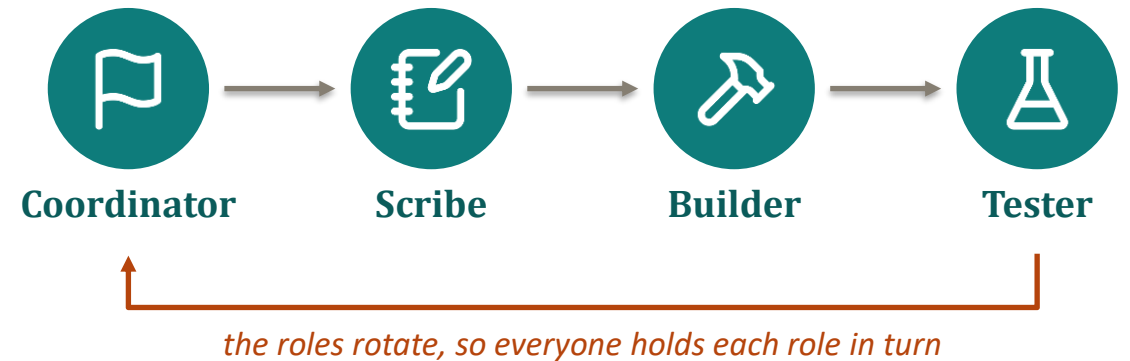
Cross-functional

Together, the team **has all the skills needed to deliver.**

Agile teams (Class 4) are small, self-organizing and cross-functional.

That is close to the **open paradigm.**

YOUR BLOCK 1 TEAM CHARTER: FOUR ROTATING ROLES



Why rotate? Everyone learns every part of the work, everyone takes part in decisions, and no single person becomes the only one who understands the code.

Stakeholders: who can affect, or is affected by, the software

A **stakeholder** is anyone who **can affect, or is affected by**, the software and its outcome.



Users

the people who use the software



Client or sponsor

pays for or sponsors the product



Operators

run it: hosting, accounts, deployment



Affected non-users

affected without ever using it



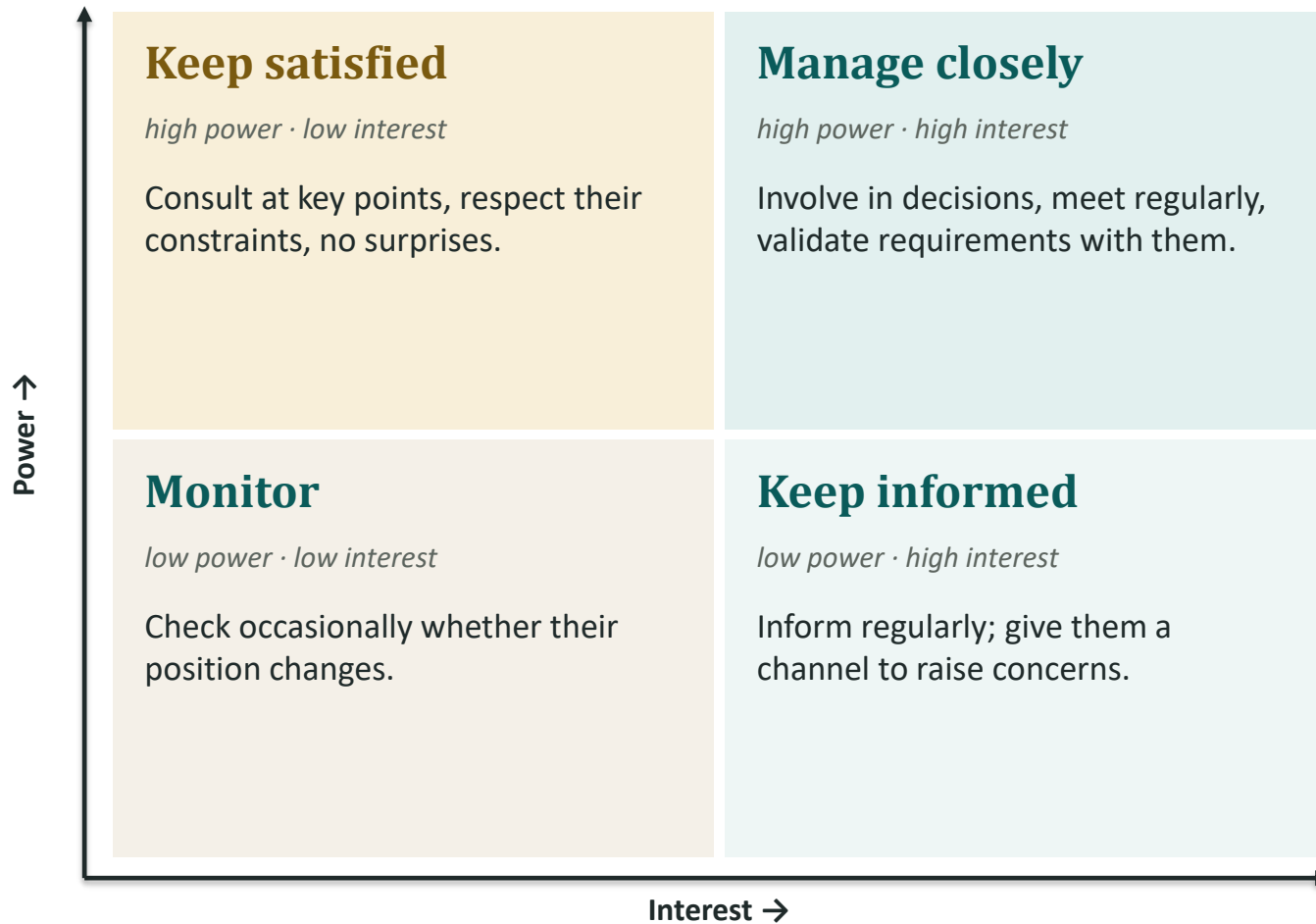
Regulators

can stop or constrain the product



The most common beginner mistake: listing only the users. Your Block 2 map needs all five groups.

The power–interest grid



Power

The stakeholder's ability to change the **requirements**, the **budget** or the **approval** of the project.



Interest

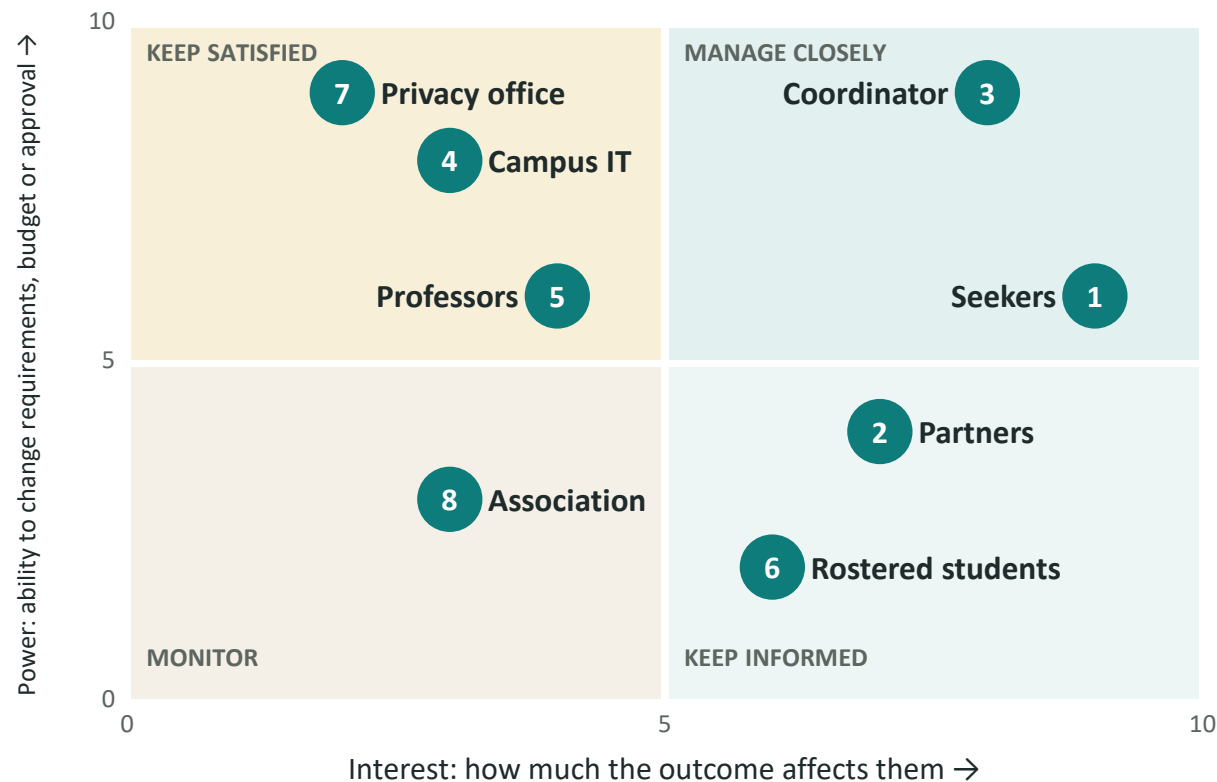
How much the **outcome** of the project affects them.

Score each stakeholder from 1 to 10 on both axes; the threshold 5 separates the four strategies (StudyBuddy, next slide).

Example — StudyBuddy stakeholders on the grid

Task Identify the stakeholders of StudyBuddy, place each on the grid, choose a strategy and a team contact, and prepare interview questions.

Relation to the lecture stakeholder definition (including people affected without using the product), the grid, rotating roles, question types.



Strategy and team contact

Manage closely

1. Students seeking partners (P6, I9) — Tester
3. Learning Centre coordinator (sponsor) (P9, I8) — Coordinator

Keep satisfied

4. Campus IT (Information Technology) operators (P8, I3) — Builder
5. Professors whose courses appear (P6, I4) — Coordinator
7. University privacy office (regulator) (P9, I2) — Coordinator

Keep informed

2. Study partners who receive requests (P4, I7) — Scribe
6. Students named in imported rosters (P2, I6) — Scribe

Monitor

8. Student association (P3, I3) — Scribe

P = power, I = interest (1–10). Rule: $P \geq 5$ and $I \geq 5 \rightarrow$ manage closely; $P \geq 5$ only $\rightarrow$ keep satisfied; $I \geq 5$ only $\rightarrow$ keep informed; otherwise monitor.

Most often forgotten: people affected without using the product (6).

Left: Figure 3 of the module, redrawn for projection.

Talking with stakeholders

COMMUNICATION PRINCIPLES (CHAPTER 6)

-  Listen.
-  Prepare before you meet.
-  Let one person facilitate.
-  Prefer face-to-face conversation.
-  Take notes and document decisions.
-  If something is unclear, draw a picture.
-  When you agree, move on; when you cannot, record the disagreement and move on.
-  Negotiate for an agreement both sides can accept — not a contest.

THREE TYPES OF INTERVIEW QUESTION

 **Open — invites a description**

“How do you find a study partner today?”

 **Closed — asks for a fact or a yes / no**

“Have you used a study-partner app before?”

 **Probing — follows up an answer for detail**

“You said it takes too long; how long, exactly?”

Class 5 Studio: prepare two of each type for every interview.



Lesson 3 after class: watch, run, reuse



Watch after class

“How to turn a group of strangers into a team”

TED (Technology, Entertainment, Design) — speaker Amy Edmondson · 12:57

Why: teaming and psychological safety — the human side of the open paradigm and of your rotating roles.

Open: [ted.com](https://www.ted.com) · [YouTube](https://www.youtube.com/watch?v=U5k1dRn3160)



Run the demo

Demo 3 — StudyBuddy stakeholders on the power–interest grid

- Places the eight stakeholders one at a time and applies the threshold rule.
- Assigns a strategy and a team contact, then prints an interview plan.
- Figure 3 and the stakeholder register (.csv).

Open it in the module (Lesson 3) or download the .html file from the Downloads section.



Comes back in your project

- **Block 2** (Oct 4): your own stakeholder map
- **Block 5** (Oct 25): six prepared questions per interview — two open, two closed, two probing
- **Lab 2** (Dec 8 → Block 12): the users become the roles in your **UAT** (User Acceptance Testing) script
- **Block 14** Showcase (Jan 12): present to the stakeholders you mapped this week

In the module (Lesson 3): five flash-cards and a self-check quiz — **formative, not graded**. Scan every link before opening it ([VirusTotal](https://www.virustotal.com)).



What a code of ethics is



A **code of ethics** is a written set of principles that a profession adopts to guide its members' decisions, especially when their own interests, or their client's, **conflict with the interests of other people**.

1999

A joint task force of the **ACM** (Association for Computing Machinery) and the **IEEE** (Institute of Electrical and Electronics Engineers) Computer Society publishes the **Software Engineering Code of Ethics and Professional Practice**



Short version

The eight principles — your reading this week.



Full version

Numbered clauses under each principle (for example 1.03, 6.07).

The commitment that opens the Code

"Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession."

The eight principles of the Code

1

PUBLIC

“Software engineers shall act consistently with the public interest.”

2

CLIENT AND EMPLOYER

“Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.”

3

PRODUCT

“Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.”

4

JUDGMENT

“Software engineers shall maintain integrity and independence in their professional judgment.”

5

MANAGEMENT

“Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.”

6

PROFESSION

“Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.”

7

COLLEAGUES

“Software engineers shall be fair to and supportive of their colleagues.”

8

SELF

“Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.”

Principle statements quoted verbatim from the [Software Engineering Code of Ethics and Professional Practice](#), © 1999 IEEE (Institute of Electrical and Electronics Engineers) and ACM (Association for Computing Machinery) (short extracts for education; the full Code may be republished only unchanged, with its notice).



PUBLIC comes first



From Figure 4: stars = the two principles that matter most for HabitHub.

Principles 2 (CLIENT AND EMPLOYER) and 6 (PROFESSION) both end with the words “**consistent with the public interest**”.



The **public interest** is the safety, health and welfare of the people affected by the software — **including people who never use it.**

When the client’s interest and the public interest conflict, the public interest comes first. That is why PUBLIC is read first.

In the active break you draw this wheel and star the two principles that matter most for your own product.

From principles to duties: selected clauses

1.03

PUBLIC

Approve software only with a well-founded belief that it is safe, meets its specifications, passes appropriate tests, and does not diminish quality of life or privacy, or harm the environment.

3.10

PRODUCT

Ensure adequate testing, debugging and review of software and related documents.

6.07

PROFESSION

Be accurate about the characteristics of your software; avoid false claims.

2.06

CLIENT AND EMPLOYER

Report promptly when a project is likely to fail, cost too much or be otherwise problematic (the late ClubLend project, Lesson 2).



Professional judgment (principle 4): your technical conclusions must not change because someone is pressuring you.

Writing “all tests pass” because the release is on Friday is a failure of judgment — even if nobody is harmed that week.

Clauses paraphrased and identified by their numbers; the first digit is the principle.

Example — HabitHub (1/2): a defect two days before release

Task Choose, and justify with the eight principles, how HabitHub should release when a defect is found two days before the release date.

Relation to the lecture the Code's principles and "the public interest comes first", applied to a release decision.

Wed.

defect D-17 found

6%

of pilot users: two devices

2 of 148

automated tests fail

Fri.

release: Wellness Week, day 1

For users who check habits on two devices, the weekly report can mark a habit as done on a day when it was not.

The sponsor (campus wellness office) wants the Friday release and release notes that say "all tests pass".

FOUR OPTIONS — VOTE BEFORE THE NEXT SLIDE

A

Release Friday and write "all tests pass" in the release notes.

B

Release Friday; switch the weekly report off for two-device users (feature flag + in-app notice); list D-17 under "Known issues"; fix in 1.0.1 within a week.

C

Delay the release by one week; fix and retest first.

D

Release Friday with the defect active; fix it quietly later; say nothing.



Example — HabitHub (2/2): the decision

HabitHub release decision: options × principles

✓ consistent · ~ tension · ✗ violation · any ✗ eliminates the option

	1	2	3	4	5	6	7	8	
A	✗	~	✗	✗	~	✗	~	~	4 ✗ 4 ~ Eliminated
B	✓	✓	~	✓	✓	✓	✓	✓	0 ✗ 1 ~ Recommended
C	✓	~	✓	✓	~	✓	✓	✓	0 ✗ 2 ~ Alternative
D	✗	✗	✗	✗	~	✗	~	~	5 ✗ 3 ~ Eliminated

Columns: 1 PUBLIC · 2 CLIENT AND EMPLOYER · 3 PRODUCT · 4 JUDGMENT
5 MANAGEMENT · 6 PROFESSION · 7 COLLEAGUES · 8 SELF

A release and write “all tests pass”

B release; report off for two-device users; known issue; fix in 1.0.1

C delay the release by one week

D release with the defect active; fix quietly; say nothing

Decision: recommend B; C is the alternative; the client decides.

From Figure 4 — Demo 4 colours the wheel for each option, one principle at a time.

Decision rule

Evaluate every option against the eight principles, PUBLIC first. Any violation eliminates the option; among the others, prefer fewer tensions.

A	4 violations, 4 tensions false claim (6.07); unmitigated defect	eliminated
D	5 violations, 3 tensions users misled; client not informed (1.04, 2.06)	eliminated
C	0 violations, 2 tensions CLIENT AND EMPLOYER; MANAGEMENT	alternative
B	0 violations, 1 tension PRODUCT: a known, mitigated defect ships	recommended

The team recommends B, presents C as the alternative, and the client decides with full information. The decision is recorded.



Licensing, certification, accreditation



Licensing

A **legal permission**, granted by a regulator, to practise a profession or to use a protected title.

Québec: the **OIQ (Ordre des ingénieurs du Québec)** regulates engineering; its stated mission is to protect the public.



Certification

A **voluntary credential** from a professional body that confirms knowledge or experience.

Example: the **I.S.P. (Information Systems Professional)** designation of **CIPS (Canadian Information Processing Society)**.



Accreditation

Applies to **programs, not to people**: the approval of an education program.

The **CEAB (Canadian Engineering Accreditation Board)** accredits engineering degree programs in Canada.



In Canada, the title “engineer” is regulated by provincial and territorial law. In Québec, as in most provinces, “engineer” — including in “software engineer” — is **reserved for licensed members** of the regulator.

Completing this course does not give that title. Rules differ between provinces and change over time: check the regulator’s own website.

General education, not legal advice.

Lesson 4 after class: watch, run, reuse



Watch after class

“Dare to disagree”

TED (Technology, Entertainment, Design) — speaker Margaret Heffernan · 12:39

Why: professional judgment — stating an inconvenient finding and defending it with evidence.

Open: [YouTube](#) · [ted.com](#)

Optional extra: “Software Engineering Ethics Explained — **IEEE CS** (Institute of Electrical and Electronics Engineers Computer Society) / **ACM** (Association for Computing Machinery) Code of Ethics” — The Programming Professor (length not published). [YouTube](#)



Run the demo

Demo 4 — HabitHub release decision on the wheel of eight principles

- Evaluates each option principle by principle and applies the rule.
- Prints an ethics decision record you can copy.
- The full Code: [computer.org](#)

Open it in the module (Lesson 4) or download the .html file from the Downloads section.



Comes back in your project

- **Block 2** (Oct 4): the “principle” column of your harms table; your 150-word reflection
- **Block 10** (Nov 29): Definition of Done and review checklist — accurate release notes (6.07)
- **Lab 3** (Dec 15 → Block 13): “Known issues” and the decision record of your Change Request
- **Final Exam** (Jan 19): Part C includes an ethics dilemma for **LO5** (Learning Outcome 5) — this example trains the method, not the answer

In the module (Lesson 4): five flash-cards and a self-check quiz — **formative, not graded**. Scan every link before opening it ([VirusTotal](#)).



Active break: visualize-it — the wheel of eight principles

0:41–0:45 · 4 minutes · on paper

- 1 Draw a circle divided into **eight segments**.
- 2 Write the eight principles **in order**, PUBLIC at the top.
- 3 **Star the two** that matter most for your own product.
- 4 Write one sentence for each: “PUBLIC matters most for our product because ...”.
- 5 Compare with your neighbour for one minute.



Model answer for HabitHub (Figure 4): PUBLIC and PROFESSION.

Five dimensions of social responsibility

Software now controls medical devices, aircraft, bank accounts and access to public services. When it fails, people can be **injured**, their information **exposed**, they can be **treated unfairly** or **excluded**, and resources can be **wasted**.



Safety

Failure must not cause death, injury or major damage.



Privacy

People control information about themselves.



Fairness

No unequal treatment of groups without a justified reason.



Accessibility

People with disabilities can perceive, understand, navigate and interact.



Sustainability

Less energy and carbon from software, on devices and in data centres.

Engineers study failures so that they are not repeated.

Safety (1/2): the Therac-25

A **safety-critical system** is a system whose failure can cause **death, serious injury, or major damage** to property or the environment.

Therac-25: a radiation therapy machine built by **AECL** (Atomic Energy of Canada Limited). 1985–1987: six known accidents with massive overdoses; some patients died. Investigation: Nancy Leveson and Clark Turner (1993). Causes that acted together:



Software alone

Earlier models had hardware interlocks, physical mechanisms that stop the machine in an unsafe state; the Therac-25 relied on software alone.



A race condition

A defect in which the result depends on the timing or order of events: fast editing of the treatment data could leave the machine inconsistent.



Error messages

Messages such as “Malfunction 54” told operators nothing useful, and operators learned to resume treatment after them.



Dismissed reports

When the first accidents were reported, the manufacturer did not accept that the machine could be the cause.

Lessons software must not be the only protection against a hazard · error messages must say what happened · reports of harm must be investigated, not dismissed.

Safety (2/2): Ariane 5 and the Boeing 737 MAX



Ariane 5, flight 501 — ESA (European Space Agency), June 4, 1996

- On its first flight, the launcher left its course and was destroyed about 40 seconds after the start of the flight sequence.
- Software reused from Ariane 4 converted a 64-bit floating-point value into a 16-bit integer; on Ariane 5's faster trajectory the value was too large, and the conversion failed.
- The backup unit ran the same software and failed the same way; the failing function was not even needed after lift-off.

Lessons

- Reused software must be verified again against the requirements of the new system.
- A backup that runs identical software does not protect against a software error.



Boeing 737 MAX — 2018 and 2019

- Lion Air 610 (October 29, 2018) and Ethiopian Airlines 302 (March 10, 2019): 346 people died.
- The **MCAS** (Maneuvering Characteristics Augmentation System) repeatedly pushed the nose down after an erroneous reading from a single angle-of-attack sensor.
- **NTSB** (National Transportation Safety Board), September 2019: the safety assessment assumed pilots would respond quickly; actual responses did not match, in part because many alerts appeared at the same time.

Lessons

- Validate assumptions about how humans will react, like any requirement.
- One sensor feeding a function that can move the flight controls is a single point of failure.

Privacy: the Desjardins breach and privacy by design

≈ 9.7 M
people affected

The Desjardins breach

2019: an employee had copied personal information from shared drives onto **USB** (Universal Serial Bus) keys over about 26 months.

- December 2020: the **OPC** (Office of the Privacy Commissioner of Canada), investigating jointly with Québec's **CAI** (Commission d'accès à l'information du Québec), found that safeguards to control access to, and transfer of, personal information were **not adequate**.
- The employee had **legitimate access**: protection from outside attackers was not enough; what matters is also who inside can see what.
- Some information was kept **far longer than needed**, without a completed retention schedule.

Every record you keep is a record that can leak: your Block 2 privacy note asks **how long** you keep each piece of data.

Privacy is a person's right to control information about themselves.

Canada: **PIPEDA** (Personal Information Protection and Electronic Documents Act) for many private-sector organizations; Québec has its own private-sector law, strengthened in 2021 by Law 25.

PRIVACY BY DESIGN: BUILD IT IN FROM THE START



Collect only the data you need



Restrict who can see it



Protect it



Delete it when no longer needed

Fairness

Fairness means that the software does not treat groups of people unequally without a justified reason. Software that makes or supports decisions about people can be unfair **even when nobody intended it**.



Evidence

Joy Buolamwini, then a researcher at the **MIT** (Massachusetts Institute of Technology) Media Lab, showed that some face-analysis systems performed worse on darker-skinned faces, and connected it to training data in which those faces were under-represented.



Policy in Canada

Under the **TBS** (Treasury Board of Canada Secretariat) Directive on Automated Decision-Making, federal institutions must complete an **Algorithmic Impact Assessment** before they use an automated decision system.

FOR YOUR PROJECT, FAIRNESS MEANS THREE THINGS



Know whom your data represents



Test the system with different groups of users



Publish the rules the system applies

Accessibility

Accessibility: people with disabilities can **perceive, understand, navigate and interact** with the software. It is a requirement like any other: specify it, design for it, test it.

The reference: **WCAG** (Web Content Accessibility Guidelines) 2.2, published by the **W3C** (World Wide Web Consortium). Four principles:

P

Perceivable

O

Operable

U

Understandable

R

Robust

Three conformance levels: **A**, **AA** and **AAA**.

Accessible Canada Act (2019)

Aims for a Canada without barriers by January 1, 2040; information and communication technologies are a priority area. It applies to organizations under federal jurisdiction — not directly to a university app — and **WCAG** remains the reference for your product.

IN PRACTICE — EACH NEED BECOMES A TEST CASE



A blind user navigates with a **screen reader**
→ **every image and every button needs a text label**



A user who cannot use a mouse
→ **every function works from the keyboard**



A deaf user
→ **captions for video**



A user with low vision
→ **sufficient contrast between text and background**

Sustainability (green software)



Sustainability in software: reducing the **energy** the software uses and the associated **carbon emissions**, on users' devices and in data centres.

The **Green Software Foundation**, a non-profit under the Linux Foundation, publishes green software practices and a metric called **SCI** (Software Carbon Intensity).

FOR A STUDENT PROJECT: SMALL, MEASURABLE CHOICES



Updates

Do not ask a server for updates every second when a **notification** will do.



Location

Do not collect location **continuously** when you need it **once**.



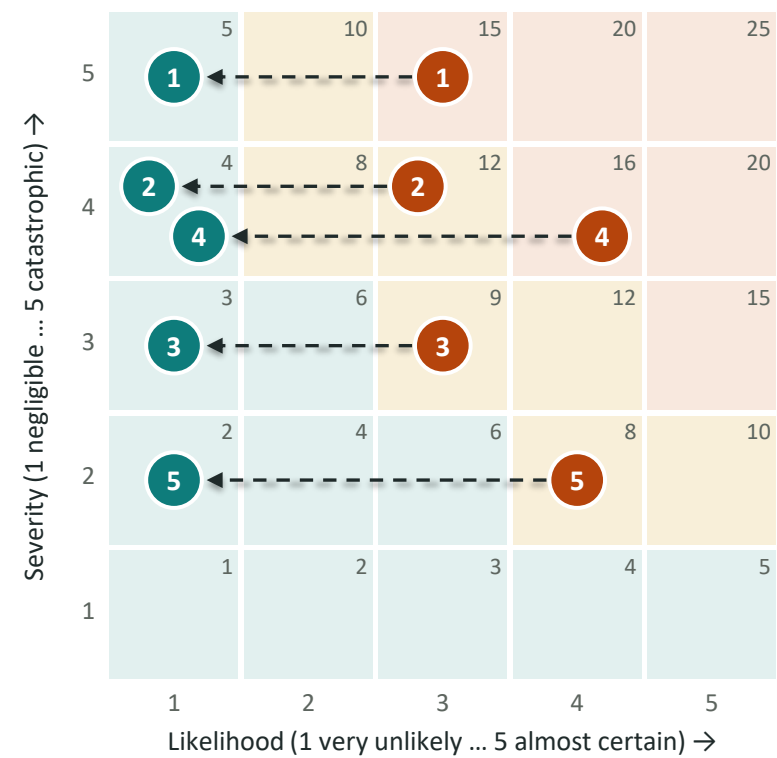
Images

Compress images before you upload them.

Example — SafeCampus: a harms register

Task Build a harms register for SafeCampus: one harm per dimension, with likelihood, severity, risk = likelihood × severity, a mitigation, and the risk that remains.

Relation to the lecture turns the five dimensions and the lessons of the cases into requirements — what Block 2 asks for your own product.



rust = before mitigation · teal = after
Figure 5 (right), redrawn for projection.

#	Dimension · harm	L × S	Mitigation	After
1	Safety urgent report unread overnight	3 × 5 = 15	Straight to campus security; emergency number on the form	5
2	Privacy anonymous reporter’s identity leaks	3 × 4 = 12	Only the safety officer sees identities; minimum fields; deleted 12 months after resolution	4
3	Fairness triage by building “importance”	3 × 3 = 9	Triage by category and time; publish the rule; monthly check	3
4	Accessibility location only by tapping a map	4 × 4 = 16	WCAG (Web Content Accessibility Guidelines) 2.2 level AA; text alternative; screen-reader test	4
5	Sustainability location read every minute	4 × 2 = 8	Location only at submission; compressed photos	2
	Total risk	60	<i>no harm left in the high zone (15–25)</i>	18

L and S from 1 to 5; low 1–7, medium 8–14, high 15–25. Block 2: at least five rows of your own, with the columns harm, who is affected, likelihood, mitigation and principle (you may add severity and risk, as here).



Closing: people knew, or could have known

Cases (rust) and responses (teal)

- 1985-87** ● **Therac-25 radiation overdoses (Atomic Energy of Canada Limited): software as the only protection**
- 1993** ● Leveson and Turner publish the Therac-25 investigation (lessons for safety-critical software)
- 1996** ● **Ariane 5 flight 501 destroyed (European Space Agency): reused software, 64-bit → 16-bit overflow**
- 1999** ● Software Engineering Code of Ethics and Professional Practice (eight principles)
- 2000** ● Personal Information Protection and Electronic Documents Act (federal privacy law)
- 2017-19** ● **Desjardins: an employee copies personal information of close to 9.7 million people**
- 2018-19** ● **Boeing 737 MAX: two crashes, 346 deaths; one angle-of-attack sensor, invalid assumptions**
- 2019** ● Safety recommendations of the National Transportation Safety Board; Accessible Canada Act; federal directive*
- 2020** ● Federal and Québec privacy commissioners publish their findings on the Desjardins breach
- 2021** ● Québec's Law 25 modernizes private-sector privacy; the Green Software Foundation is founded
- 2023** ● Web Content Accessibility Guidelines 2.2 published as a World Wide Web Consortium standard

* Treasury Board of Canada Secretariat Directive on Automated Decision-Making.

From Figure 5 of the module (download it for a closer look).

In each case today, people knew, or could have known, that something was wrong **before the harm occurred**.

The professional response: report what you know, and design the system so that the harm cannot happen.

Each of these failures also led to standards, laws and review practices — and you can apply them from your first Block.

LO5 (Learning Outcome 5) — recognise the professional, ethical and social responsibility of a Software Engineer.



Lesson 5 after class: watch, run, reuse



Watch after class

“How I’m fighting bias in algorithms”

TED (Technology, Entertainment, Design) — speaker Joy Buolamwini · 8:34

Open: [YouTube](#) · [ted.com](https://www.ted.com)

“Introduction to Web Accessibility and W3C (World Wide Web Consortium) Standards”

W3C Web Accessibility Initiative · about 4 minutes

Open: [YouTube](#) · [w3.org](https://www.w3.org)

“Case Study 1 — Introduction and Therac-25 (AnMei Little)”

Yale / Coursera, “Introduction to Medical Software” · about 9 minutes

Open: [YouTube](#) · [coursera.org](https://www.coursera.org)

Optional extra: “Two Boeing 737 Max Crashes and a ‘Fatal Flaw’” — FRONTLINE PBS (Public Broadcasting Service) | Official (an excerpt; length not published). [YouTube](#)



Run the demo

Demo 5 — SafeCampus harms register on a 5 × 5 risk matrix

- Places each harm and computes risk = likelihood × severity.
- Applies the mitigation and totals the risk before (60) and after (18).
- Figure 5 and the harms register (.csv).

Open it in the module (Lesson 5) or download the .html file from the Downloads section.



Comes back in your project

- **Block 2** (Oct 4): your harms table (at least 5 rows) and your data and privacy note
- **Block 5** (Oct 25): each mitigation becomes a requirement — **NFR** (Non-Functional Requirement) entries for security, privacy and usability
- **Lab 2** (Dec 8 → Block 12): an accessibility scenario in your **UAT** (User Acceptance Testing) script
- **Lab 3** (Dec 15 → Block 13): harms not fully mitigated go under “Known issues”
- **Final Exam** (Jan 19): harm → people affected → principle → mitigation

In the module (Lesson 5): five flash-cards and a self-check quiz — **formative, not graded**. Scan every link before opening it ([VirusTotal](#)).



Active break: Zen-break, then “Who was responsible?”

0:56–1:00 · 4 minutes



1 · Zen-break — two minutes

Silence.

Breathe slowly.

Stretch.

Look away from the screen.



2 · Think-pair-share: “Who was responsible?”

Pick Therac-25, Ariane 5 or the 737 MAX.

1 minute alone: list everyone who had a part in the outcome — programmers, testers, managers, the company, the regulator, the users.

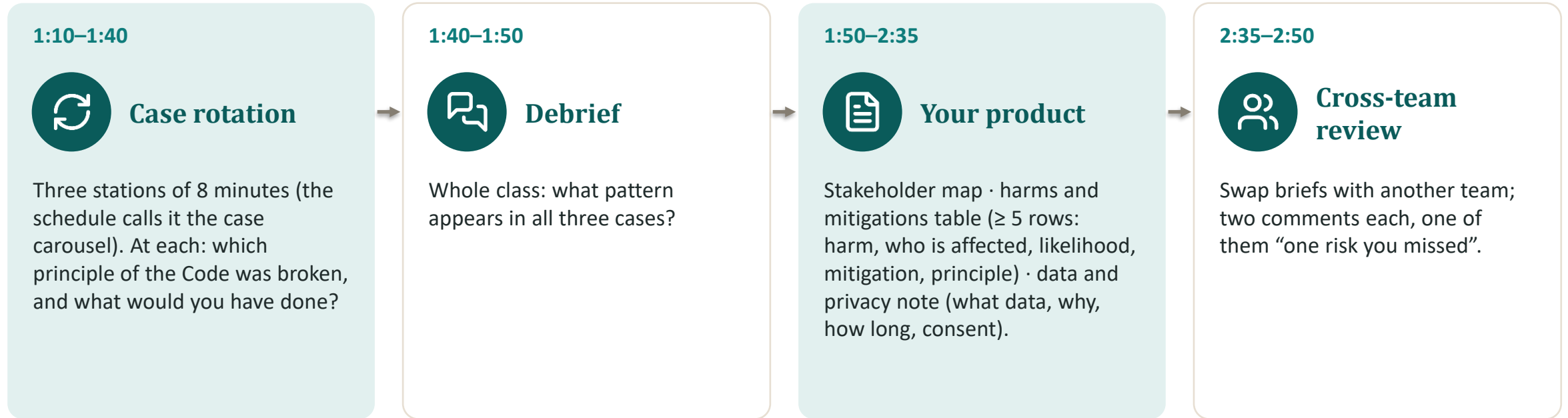
1 minute in pairs: which principle of the Code applied to each of them?

Then three students report to the class.

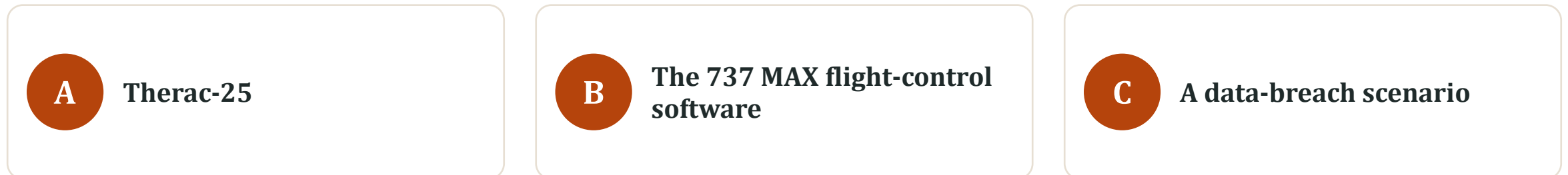
The exit ticket comes at the end of the Studio (2:50). Break at 1:00; Studio at 1:10.



Studio (100 minutes): judge three cases, then brief your product



CASE ROTATION: THREE STATIONS



Block 2 — Stakeholder & Ethics Brief (summary)



Stakeholder map

users, client, operators, affected third parties, regulators



Harms and mitigations

a table of at least five rows: harm, who is affected, likelihood, mitigation, principle



Data and privacy note

what data, why, how long, consent



Reflection (150 words)

“The engineer I want to be” — personal

Where it goes next: the privacy, security and accessibility requirements of Block 5, and the ethics question of the Final Exam, which assesses **LO5** (Learning Outcome 5).

Sun., Oct 4 · 23:59

due (course local time)

2%

of the course grade

1

submission per team, on the **LMS** (Learning Management System)

- File `/docs/02-ethics-brief.md` in your team repository, tag **block2**; submit the repository link and the tag.
- The complete brief and its rubric are published separately: this is a summary.

Your team’s own work: declare any **AI** (Artificial Intelligence) tool you used, be ready to explain every line, fictional data only ([Legal Notice, s. 18](#)).

Wrap-up: exit ticket and Home Sprint



Exit ticket — one sentence

Which principle of the Code will your product test the most — that is, which principle will be hardest to respect in your product? Write one sentence: the principle, and the feature of your product that makes it hard to respect.

HOME SPRINT (120 MINUTES)

- Finish Block 2, including the 150-word reflection “The engineer I want to be”.
- Read Pressman and Maxim, 9th edition, Chapter 5 (Human Aspects of Software Engineering) and Chapter 6 (Principles That Guide Practice).
- Read the short version of the Software Engineering Code of Ethics and Professional Practice (link on the Links slides).
- Flash-cards: add this week’s nine terms to your deck and review last week’s.

FLASH-CARD TERMS OF THE WEEK

software myth

Brooks’s law

stakeholder

code of ethics

public interest

safety-critical system

privacy by design

accessibility

professional judgment



Links (1/3)

Every link opens a third-party site. Scan it first: [VirusTotal](#) or [Google Safe Browsing](#) ([Legal Notice](#), [Annex C](#)). A clean result is never a guarantee.

TERMS AND LINK SAFETY

[Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy \(v2.0\)](#) — The terms that govern this module and every file of this lecture

[VirusTotal — scan a link](#) — Paste any link here to check its reputation before you open it (Annex C of the Legal Notice)

[Google Safe Browsing — site status](#) — Checks whether a domain is currently flagged as unsafe

COURSE DOCUMENTS

[CS \(Computer Science\) 330 class-by-class schedule](#) — Where Class 2 fits in the semester, the Blocks, the Labs and the grading (course file; same folder as the module)

[Class 1 lecture module](#) — Last week's module (course file; same folder as the module)

CODES OF ETHICS AND PROFESSIONAL BODIES

[Software Engineering Code of Ethics and Professional Practice \(IEEE \(Institute of Electrical and Electronics Engineers\) Computer Society\)](#) — The full text of the Code of Lesson 4

[Software Engineering Code of Ethics on the ACM \(Association for Computing Machinery\) site](#) — The same Code, published by the ACM

[ACM Code of Ethics and Professional Conduct \(2018\)](#) — The general code of the ACM for all computing professionals

[SWEBOK \(Software Engineering Body of Knowledge\) Guide, version 4.0 \(IEEE Computer Society\)](#) — The Guide to the Software Engineering Body of Knowledge (2024)

[Ordre des ingénieurs du Québec \(English site\)](#) — The regulator of the engineering profession in Québec

[Engineers Canada — use of the title “engineer”](#) — Who may use “engineer” and P.Eng. (Professional Engineer) in Canada, the 12 provincial and territorial regulators, and the 2023 Alberta change for “software engineer”

[Engineers Canada \(home of the CEAB \(Canadian Engineering Accreditation Board\)\)](#) — National organization of the engineering regulators



Links (2/3)

Every link opens a third-party site. Scan it first: [VirusTotal](#) or [Google Safe Browsing](#) ([Legal Notice](#), [Annex C](#)). A clean result is never a guarantee.

CODES OF ETHICS AND PROFESSIONAL BODIES

[CIPS — Canadian Information Processing Society](#) — Canadian professional association for IT (Information Technology)

[CIPS — I.S.P. \(Information Systems Professional\) certification](#) — How the Information Systems Professional designation is obtained (linked from the **CIPS** home page)

[NCEES \(National Council of Examiners for Engineering and Surveying\) — discontinuing the PE \(Professional Engineer\) Software Engineering exam](#) — The United States examination body's announcement that the **PE** exam in software engineering ended after April 2019

BOOKS AND PEOPLE

[George Pólya, How to Solve It \(Princeton University Press\)](#) — The 1945 book behind the four steps of the essence of practice (Lesson 1)

[Frederick P. Brooks, Jr. — ACM \(Association for Computing Machinery\) Turing Award \(1999\)](#) — Biography of the author of The Mythical Man-Month and of Brooks's law (Lesson 2)

[THUNK, episode 182 — Lessons From “The Mythical Man-Month”](#) — The episode page of the second Lesson 2 video

CASES

[Ariane 501 — presentation of the Inquiry Board report \(ESA \(European Space Agency\)\)](#) — ESA's official summary of why the first Ariane 5 was lost on 4 June 1996 (Lesson 5)

[NTSB \(National Transportation Safety Board\) Safety Recommendation Report ASR-19-01 \(Boeing 737 MAX\)](#) — Assumptions used in the safety assessment and the effect of multiple alerts on pilots (September 19, 2019)

[United States House committee — final report on the Boeing 737 MAX \(2020\)](#) — The 18-month congressional investigation of the design, development and certification of the 737 MAX

[OPC \(Office of the Privacy Commissioner of Canada\) findings PIPEDA-2020-005 — Desjardins breach](#) — The joint investigation with the **CAI** (Commission d'accès à l'information du Québec) (December 14, 2020)

[Leveson and Turner, An Investigation of the Therac-25 Accidents \(author's copy, MIT \(Massachusetts Institute of Technology\)\)](#) — The 1993 investigation that the Therac-25 part of Lesson 5 summarizes

[Coursera — Introduction to Medical Software \(Yale University\)](#) — The course that includes the Therac-25 case-study video of Lesson 5

LAWS, POLICIES AND STANDARDS

[PIPEDA \(Personal Information Protection and Electronic Documents Act\) — overview \(Office of the Privacy Commissioner of Canada\)](#) — The federal private-sector privacy law, explained by its regulator



Links (3/3)

Every link opens a third-party site. Scan it first: [VirusTotal](#) or [Google Safe Browsing](#) ([Legal Notice](#), [Annex C](#)). A clean result is never a guarantee.

LAWS, POLICIES AND STANDARDS

[Commission d'accès à l'information du Québec](#) — Québec's access-to-information and privacy regulator, which oversees Law 25

[Accessible Canada Act \(S.C. \(Statutes of Canada\) 2019, c. 10\)](#) — The federal law that aims for a Canada without barriers by January 1, 2040

[Directive on Automated Decision-Making \(Treasury Board of Canada Secretariat\)](#) — The federal policy that requires an impact assessment before an automated decision system is used

[Algorithmic Impact Assessment tool \(Canada.ca\)](#) — The questionnaire (65 risk and 41 mitigation questions) that sets impact levels I to IV under the Directive

[WCAG \(Web Content Accessibility Guidelines\) 2.2 — W3C \(World Wide Web Consortium\) Recommendation](#) — The accessibility standard of Lesson 5

[W3C WAI \(Web Accessibility Initiative\) — video "Introduction to Web Accessibility and W3C Standards"](#) — The official page of the accessibility video of Lesson 5, with transcript and captions

[Green Software Foundation](#) — Non-profit under the Linux Foundation

[Green Software Practitioner course](#) — A free introductory course on green software (linked from the Foundation's home page)

TALKS (OFFICIAL PAGES)

[Tom Wujec — Got a wicked problem? First, tell me how you make toast \(TED \(Technology, Entertainment, Design\)\)](#) — Lesson 1 video, 8:51

[Amy Edmondson — How to turn a group of strangers into a team \(TED\)](#) — Lesson 3 video, 12:57

[Margaret Heffernan — Dare to disagree \(TED\)](#) — Lesson 4 video, 12:39

[Joy Buolamwini — How I'm fighting bias in algorithms \(TED\)](#) — Lesson 5 video, 8:34



Legal notice: how it applies to CS 330

CS (Computer Science) 330 is a software engineering course, not a cybersecurity course. The Cyber.SoHo Notice applies with these adaptations: “Authorized Lab Environment” = your own computer and your team’s course repository, with fictional data only; “the College” = the University. In case of doubt, the full [Notice](#) governs.

 Education only s. 3	 The Notice’s own code of ethics s. 9
 Your licence: one personal copy for study; short quotes with attribution s. 4.2	 Demos “as is”; you are responsible for how you reuse the code s. 10–11
 Do not repost, share, sell or upload the materials, or use them to train models s. 4.3	 Links and videos: third-party sites; scan first; platform terms apply s. 14–15
 The Cyber.SoHo name, logo and slogan are trademarks s. 4.4	 Laws and licensing rules: general education, not legal advice s. 16
 Code of Ethics principles quoted verbatim, with their notice s. 4.6	 No recording without permission; quiz solutions are confidential s. 17
 Scenarios and their numbers are fictional teaching examples s. 4.7	 Academic integrity: declare AI (Artificial Intelligence) tools; fictional data only s. 18
 Third-party names for identification only; no affiliation s. 5	 Personal information: the module collects nothing s. 20